WEEK 1 · DAY 4 · OVERVIEW

Classes

A Bond Object



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 8 SECTIONS / 6 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Sixth Note

Every day we hand current_yield a coupon and a price, and on Day 3 a price of zero walked straight in. Build me a Bond that carries its own terms, works out its own numbers, refuses a bad price, and prints in words. Then tie the book's value and income from four Bonds.

ABOUT 50 MIN WITH EXERCISES / 8 SECTIONS / 6 EXERCISES

Your Route Through Day 4

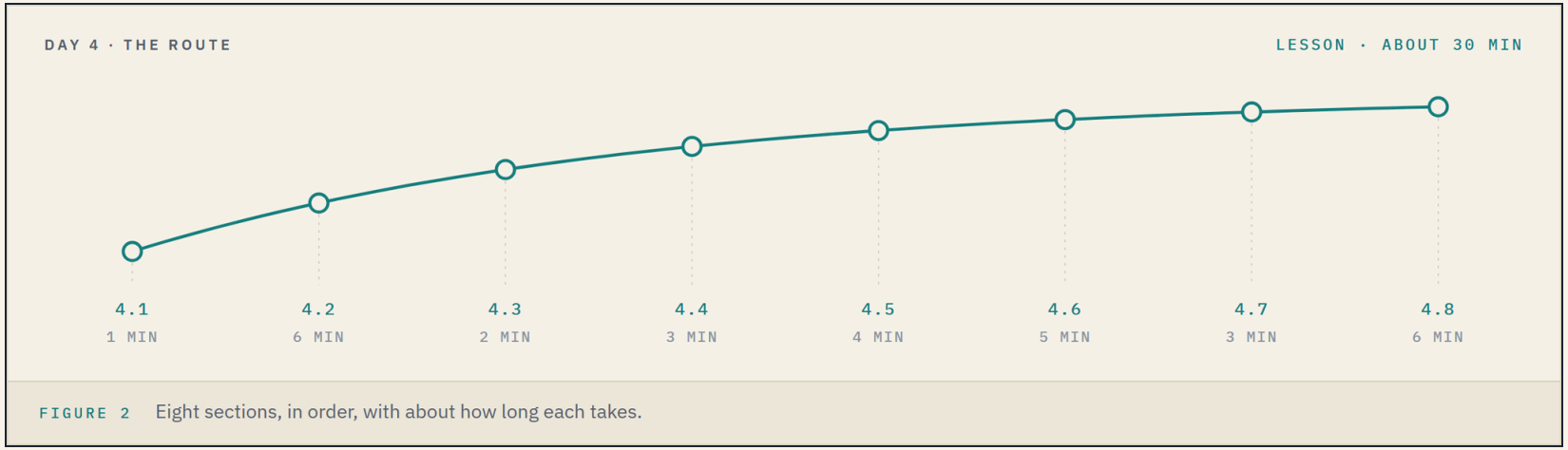


FIGURE 2 Eight sections, in order, with about how long each takes.

	SECTION	ABOUT
4.1	Objects Are Already Everywhere	1 min
4.2	A First Class	6 min
4.3	The Trouble With Loose Attributes	2 min
4.4	The Initializer	3 min

	SECTION	ABOUT
4.5	Working With an Object	4 min
4.6	A Bond That Refuses a Bad Price	5 min
4.7	Printing an Object	3 min
4.8	A Portfolio of Objects	6 min

Time. About 30 minutes for the lesson, then about 18 for the six exercises.

4.1 Objects Are Already Everywhere

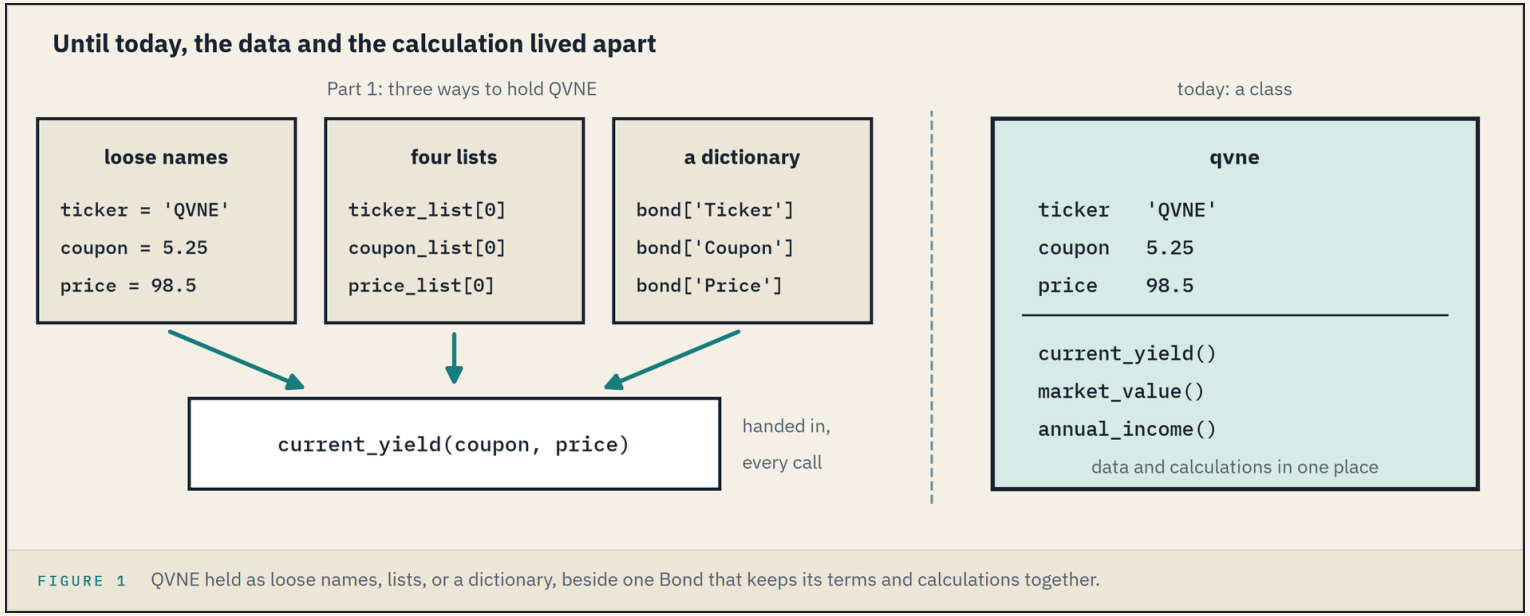


FIGURE 1 QVNE held as loose names, lists, or a dictionary, beside one Bond that keeps its terms and calculations together.

THE CODE

```
coupon = 5.25
ticker = 'QVNE'
print(type(coupon))
print(type(ticker))
print(ticker.lower())
```

```
<class 'float'>
<class 'str'>
qvne
```

Python's own classes cover numbers and text. Now write one for a bond.

4.2 A First Class

A FIRST CLASS

```
class Bond:
    """A bond: its terms, and its current yield."""
    def set_terms(self, ticker, coupon, price):
        self.ticker = ticker
        self.coupon = coupon
        self.price = price
    def current_yield(self):
        return self.coupon / self.price * 100
```

MAKE ONE, FILL IT IN

```
qvne = Bond()
qvne.set_terms('QVNE', 5.25, 98.5)
print(type(qvne))
```

```
<class '__main__.Bond'>
```

ASK IT

```
print(qvne.coupon)
print(qvne.current_yield())
```

```
5.25
5.32994923857868
```

A class is a blank form until we fill one in.

What self Means

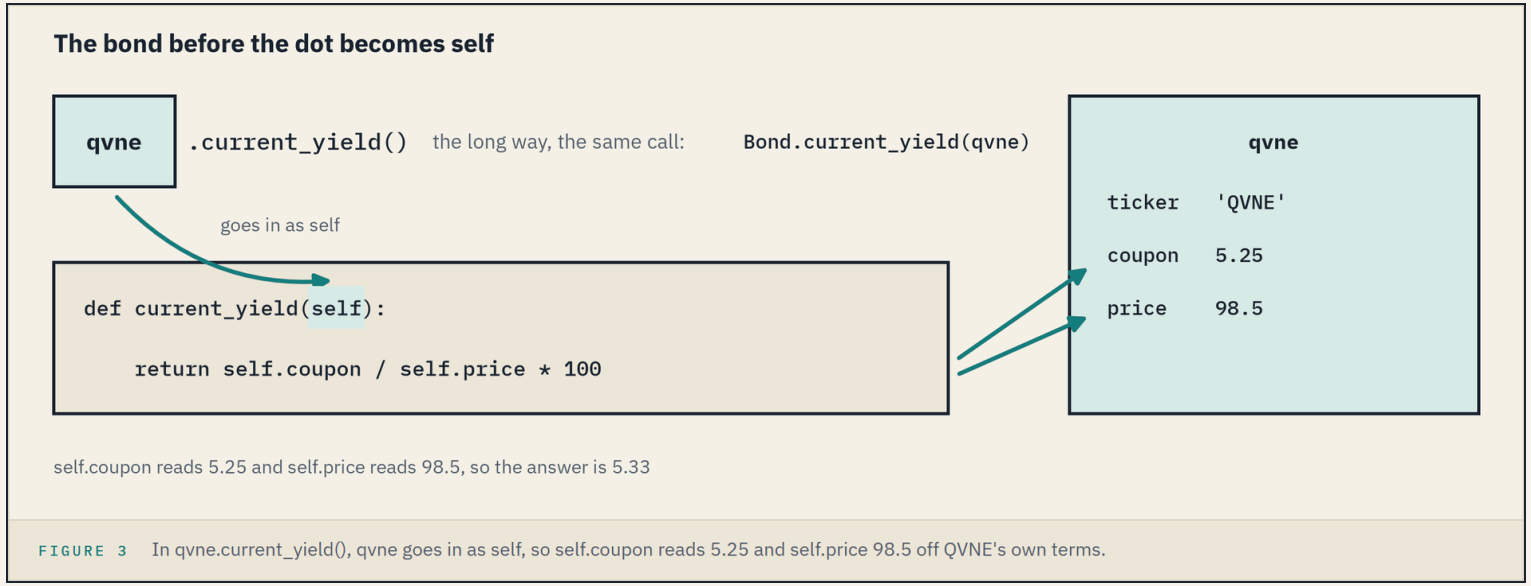


FIGURE 3 In qvne.current_yield(), qvne goes in as self, so self.coupon reads 5.25 and self.price 98.5 off QVNE's own terms.

THE LONG WAY

```
print(Bond.current_yield(qvne))
5.32994923857868
```

OUTSIDE THE CLASS

```
print(self.coupon)
NameError: name 'self' is not defined
```

Inside the class `self` is this bond; outside, use the bond's own name.

4.3 The Trouble With Loose Attributes

A SECTOR ON ONE BOND ONLY

```
qvne.sector = 'Energy'
print(qvne.sector)
```

```
Energy
```

THE NEXT BOND

```
print(dnmr.sector)
```

```
AttributeError: 'Bond' object has no attribute 'sector'
```

A BOND NEVER FILLED IN

```
plwk = Bond()
print(plwk.coupon)
```

```
AttributeError: 'Bond' object has no attribute 'coupon'
```

Every Bond needs the same terms from the moment it is made.

4.4 The Initializer

One class, four objects: the terms differ, the methods are shared

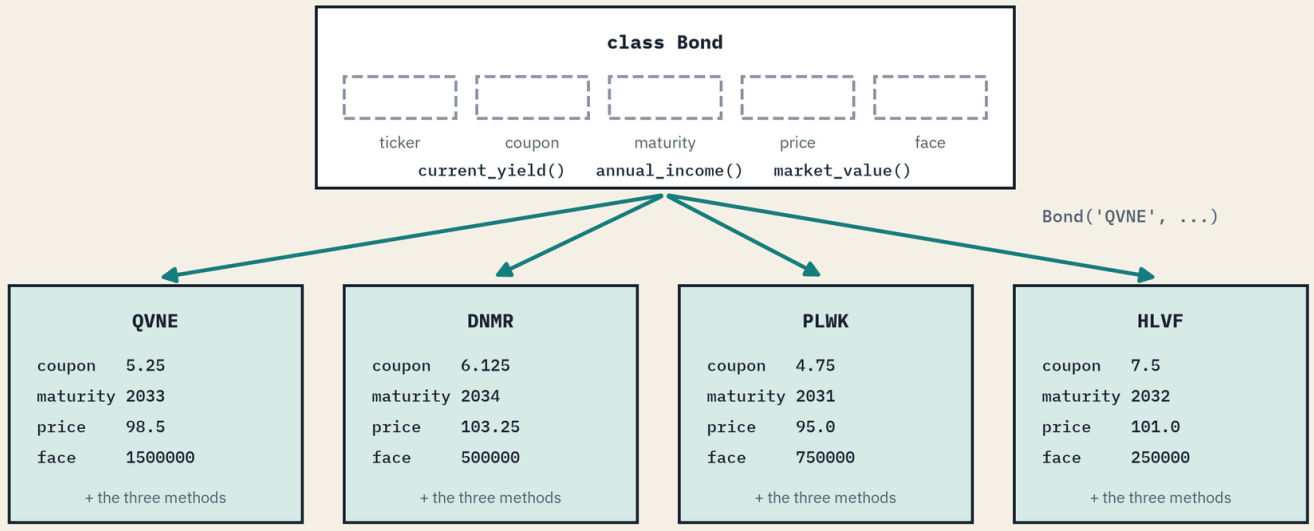


FIGURE 4 One Bond class, five terms and three methods, makes four objects, each with its own terms and the same methods.

FOUR BONDS, ONE LINE EACH

```
qvne = Bond('QVNE', 5.25, 2033, 98.5, 1500000)
dnmr = Bond('DNMR', 6.125, 2034, 103.25, 500000)
plwk = Bond('PLWK', 4.75, 2031, 95.0, 750000)
hlvf = Bond('HLVF', 7.5, 2032, 101.0, 250000)
print(plwk.coupon)
```

```
4.75
```

The initializer will not make a half-filled Bond.

4.5 Working With an Object

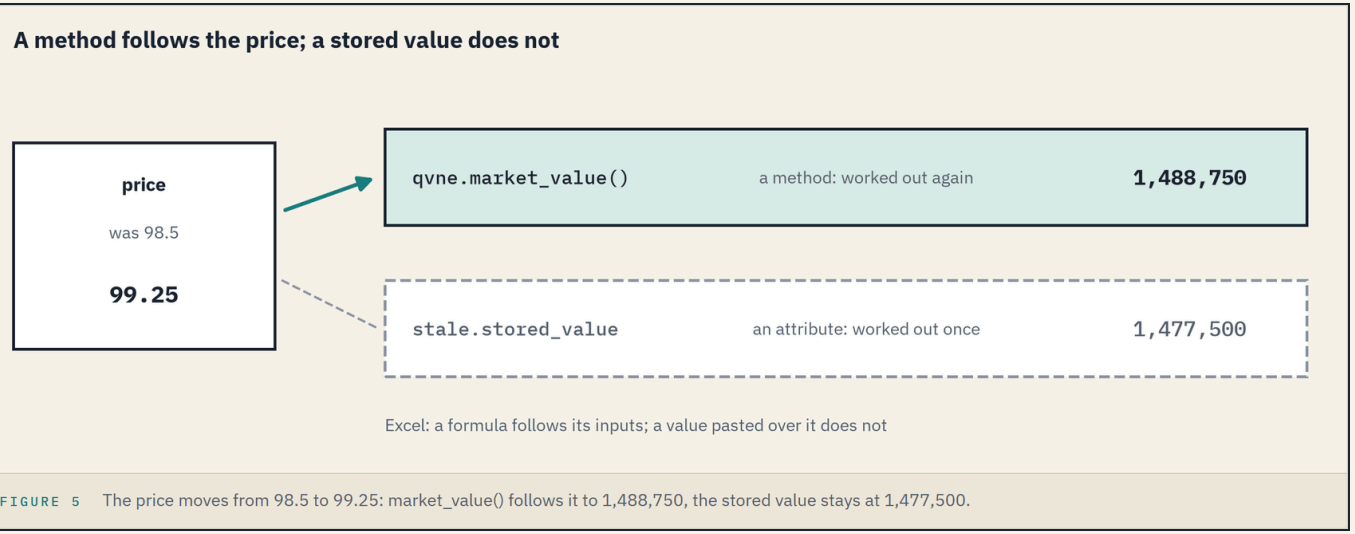


FIGURE 5 The price moves from 98.5 to 99.25: market_value() follows it to 1,488,750, the stored value stays at 1,477,500.

THE METHOD FOLLOWS THE PRICE

```
print(f'Market value: ${qvne.market_value():,.2f}')
```

```
Market value: $1,477,500.00
```

A STORED VALUE GOES STALE

```
stale = BondWithStoredValue('QVNE', 98.5, 1500000)
print(f'Before the mark: ${stale.stored_value:,.2f}')
print(f'After the mark:  ${stale.stored_value:,.2f}')
```

```
Before the mark: $1,477,500.00
After the mark:  $1,477,500.00
```

Store the facts as attributes; calculate anything that depends on them in a method.

4.6 A Bond That Refuses a Bad Price

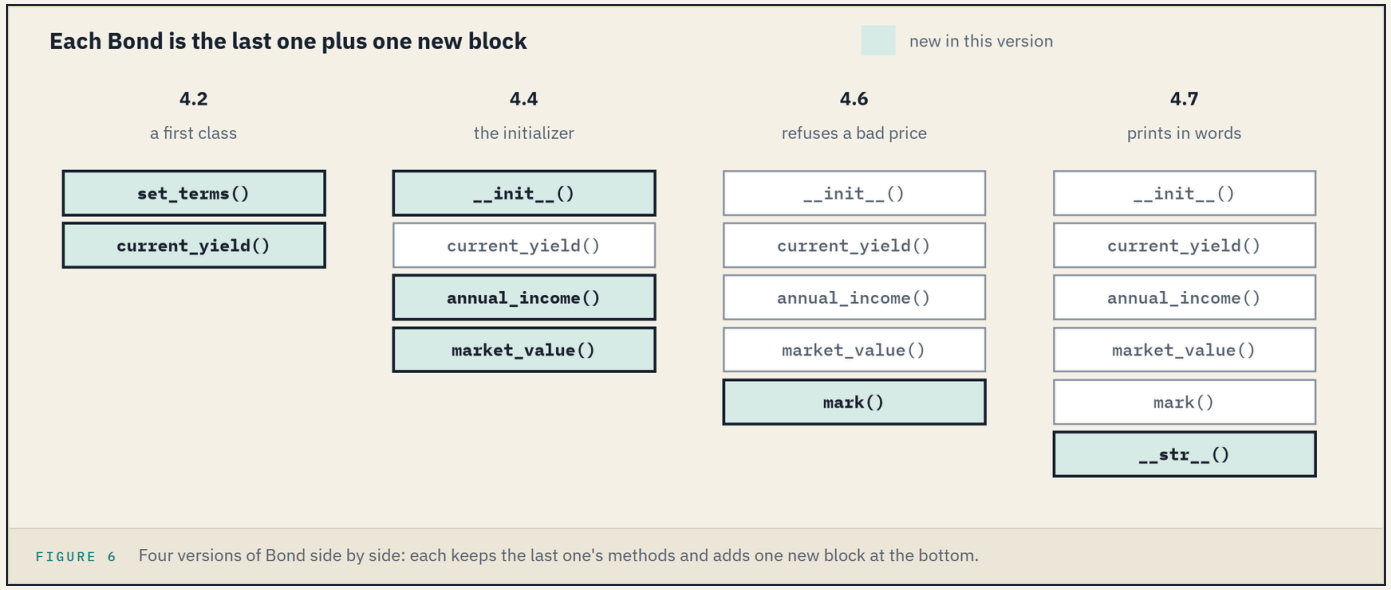


FIGURE 6 Four versions of Bond side by side: each keeps the last one's methods and adds one new block at the bottom.

MARK IT, THEN TRY ZERO

```
qvne.mark(99.25)
qvne.mark(0)
```

```
ValueError: the price must be above zero, and it is 0
```

THE REFUSED PRICE NEVER ARRIVED

```
print(qvne.price)
```

```
99.25
```

The Bond refuses a bad price itself: Day 3's `raise`, now inside the class.

4.7 Printing an Object

WHAT PRINT SHOWS

```
print(qvne)
```

```
<__main__.Bond object at 0x0000023387F2ABA0>
```

__STR__ GIVES IT WORDS

```
qvne = Bond('QVNE', 5.25, 2033, 98.5, 1500000)
print(qvne)
```

```
QVNE 5.25% of 2033, current yield 5.33%
```

PYTHON CALLS THESE ITSELF

METHOD	PYTHON CALLS IT WHEN
`__init__`	a Bond is made
`__str__`	a Bond is printed
`__repr__`	a list of Bonds is printed; not written today

EXCEL SIDE BY SIDE

Excel side by side. `__str__` builds a label from the facts, as a formula does: with ticker, coupon, and maturity in A2:C2, `=A2&" "&B2&"% of "&C2` returns QVNE 5.25% of 2033.

Now a Bond prints in words.

4.8 A Portfolio of Objects

A list of Bonds reads like a holdings sheet

attributes: plain columns — methods: formula columns

the class →

Ticker	Coupon	Maturity	Price	Face	Current yield	Market value	Income
QVNE	5.25	2033	98.5	1,500,000	5.33	1,477,500	78,750
DNMR	6.125	2034	103.25	500,000	5.93	516,250	30,625
PLWK	4.75	2031	95.0	750,000	5.00	712,500	35,625
HLVF	7.5	2032	101.0	250,000	7.43	252,500	18,750

each row: one object

fill down = the loop

FIGURE 7 The four bonds as a sheet: plain columns are attributes, formula columns are methods, each row is an object.

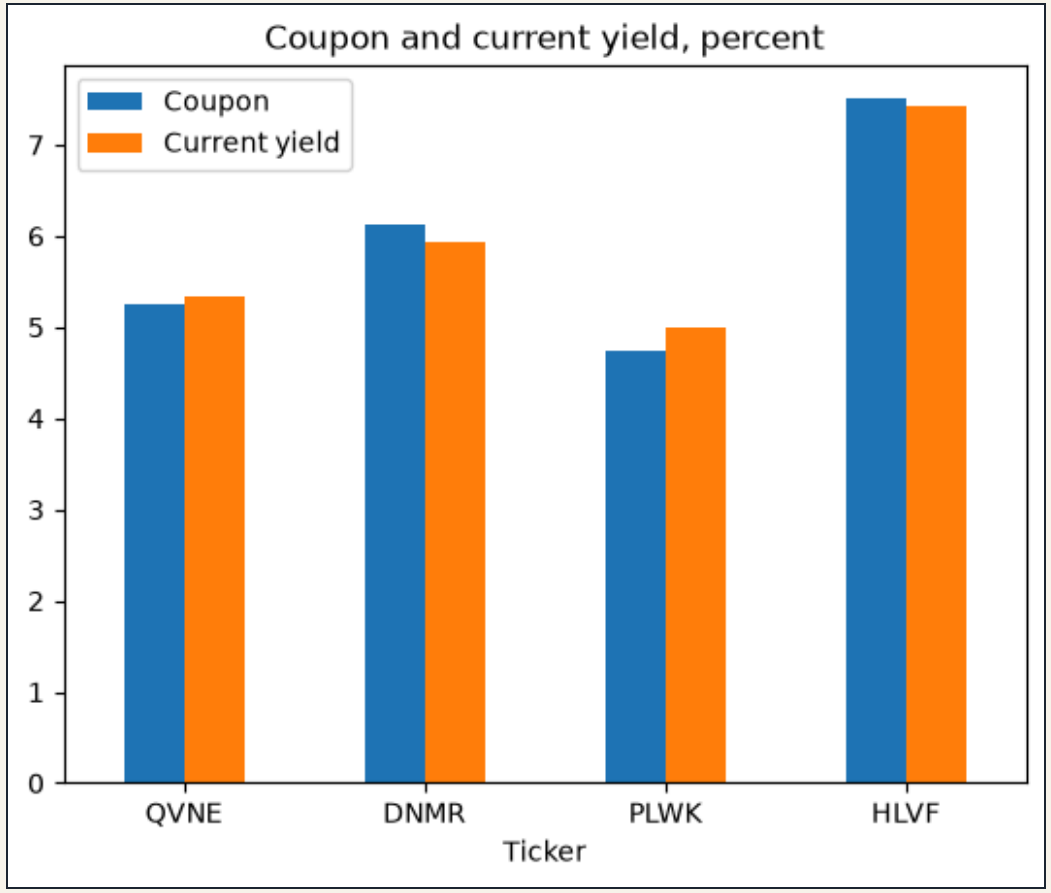
THE BOOK IS A LIST

```
portfolio = [qvne, dnmr, plwk, hlvf]
for bond in portfolio:
    print(bond)
```

```
QVNE 5.25% of 2033, current yield 5.33%
DNMR 6.125% of 2034, current yield 5.93%
PLWK 4.75% of 2031, current yield 5.00%
HLVF 7.5% of 2032, current yield 7.43%
```

Each bond carries everything it needs.

Discounts and the Book



EACH BOND, ASKED

	TICKER	COUPON	CURRENT YIELD
0	QVNE	5.250	5.329949
1	DNMR	6.125	5.932203
2	PLWK	4.750	5.000000
3	HLVF	7.500	7.425743

THE BOOK TIES

```
book_value = sum([bond.market_value() for bond in portfolio])
book_income = sum([bond.annual_income() for bond in portfolio])
```

```
Book ties: $2,958,750.00 value, $163,750.00 income
```

Below par, the current yield is above the coupon; above par, below it.

You Can Now

- Read a class: its attributes, its methods, and **`self`**.
- Make a **Bond** in one line with an initializer.
- Store the facts as attributes and calculate the rest in methods.
- Make a method refuse a bad value with **`raise`**.
- Make an object print in words a person can read.
- Loop over a list of objects.

THE NOTE ASKED	ANSWER
Its own terms and numbers	`current_yield()`, `market_value()`, `annual_income()`, nothing handed in
Refuse a bad price	`qvne.mark(0)` stops with a **`ValueError`** that says why
Print in words	`QVNE 5.25% of 2033, current yield 5.33%`
The book from four Bonds	two **`assert`** lines tie the value and the income

NEXT
WEEK 1 · DAY 5

Case Study: Organizing a Small Portfolio

One job from start to finish, using everything from the week.